

ABC 284 补题报告

E题

感觉是dfs一下连通块用公式求，没时间写了

一开始想用连通块，发现并不知道有啥公式，直接dfs一遍就行

```
vector<int> a[N];
bool st[N];
int res;
void dfs(int u)
{
    if(res>=1e6||st[u]==true)
    {
        return ;
    }
    st[u]=true;
    res++;
    for(auto y:a[u])
    {
        dfs(y);
    }
    st[u]=false;
}
signed main()
{
    int n,_;
    scanf("%d%d",&n,&_);
    while (_--)
    {
        int p,q;
        scanf("%d%d",&p,&q);
        a[p].push_back(q);
        a[q].push_back(p);
    }
    dfs(1);
    printf("%d\n",res);
    return 0;
}
```

F题

主要就是判断一个长度为n的字符串，与前半段和后半段的和是否为回文。

应为要遍历整个字符串，这其中涉及到多次查询，字符串hash法可以做到O(1) 查询

P数组主要是作为换算的数组，利用h1和h2正反两次求hash刚好可以很方便的判断，顺便还能减少hash冲突。

```

#include <bits/stdc++.h>
typedef long long ll;
using namespace std;
const int N=2e6+10;
const ll p=13331,mod=998244353;
char s[N];
ll P[N],h1[N],h2[N];
signed main()
{
    int n;
    scanf("%d",&n);
    scanf("%s",s+1);
    P[0]=1;
    for(int i=1;i<=2*n;i++)
    {
        h1[i]=(h1[i-1]*p%mod+s[i]-'a'+mod)%mod;
        P[i]=P[i-1]*p%mod;

    }
    for(int i=n*2;i>=1;i--)
        h2[i]=(h2[i+1]*p%mod+s[i]-'a'+mod)%mod;
    for(int i=0;i<=n;i++)
    {
        ll t1=(h2[i+1]-h2[i+n+1]*P[n]%mod+mod)%mod;
        ll t2=(h1[i]*P[n-i]%mod+h1[2*n]-h1[i+n]*P[n-i]%mod+mod)%mod;
        if(t1==t2)
        {
            for(int j=i+n;j>=i+1;j--)
            {
                printf("%c",s[j]);
            }
            printf("\n%d\n",i);
            return 0;
        }
    }
    puts("-1");
    return 0;
}

```

附加（线性dp）vj第四周D题

这里表示移动或者不移动的情况，这里设为j一直++的原因是在遍历i的时候，需要从小到大算出移动的次数，若最多运动j次的apple数大于j-1次其实就代表了当前状态下不用移动 对于每个不同的i来说其实情况都不一样，相当于于是i不变，从能移动0到k次进行遍历

j代表最多能够移动j次，而不是必须，dp[i]转移时候是从i-1的0到k移动的情况取得最大值

```

#include <cstdio>
#include <cstdlib>
#include <iostream>
#include <algorithm>
using namespace std;
const int N=1000+50;

```

```

int dp[N][50];
int a[N];
signed main()
{
    int n,k;
    scanf("%d%d",&n,&k);
    for(int i=1;i<=n;i++)
    {
        scanf("%d",&a[i]);
    }
    for(int i=1;i<=n;i++)
    {
        for(int j=0;j<=k;j++)
        {
            if(j==0) dp[i][j]=dp[i-1][j];
            else
                dp[i][j]=max(dp[i-1][j],dp[i-1][j-1]);
            if(j%2==a[i]-1) dp[i][j]++;
        }
    }
    int res=dp[n][0];
    for(int i=1;i<=k;i++) res=max(res,dp[n][i]);
    printf("%d\n",res);
    return 0;
}

```